

A Mini Project Report on

**“Crime Pattern Detection, Analysis & Prediction Through
Machine Learning”**

Submitted to the

JAWAHARLAL NEHRU TECHNOLOGICAL UNIVERSITY HYDERABAD

In partial fulfillment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

**COMPUTER SCIENCE & ENGINEERING
(Artificial Intelligence and Machine Learning)**

BY

MIRZA YASSER ALI BAIG (21WJ1A6642)

SYED AFFAN AHMED (21WJ1A6653)

MOHAMMED SHAHNAWAZ (22WJ5A6603)

Under the Esteemed Guidance Of

Dr. S. Madhu,

Professor, CSE (AIML), GNITC



**Department of Computer Science Engineering
(Artificial Intelligence and Machine Learning)**

GURU NANAK INSTITUTIONS TECHNICAL CAMPUS (AUTONOMOUS)

School of Engineering and Technology,

Ibrahimpattam, R.R. District 501506,

2024-2025



GURU NANAK INSTITUTIONS TECHNICAL CAMPUS



Approved by
AICTE - New Delhi



Affiliated to
JNTU - Hyderabad



Accredited by
National Assessment and
Accreditation Council



Accredited by
National Board of
Accreditation

AUTONOMOUS
under Section 2 (f) & 12 (b) of
University Grants Commission Act

Department of Computer Science & Engineering (Artificial Intelligence and Machine Learning)

CERTIFICATE

This is to certify that this minor project entitled “**Crime Pattern Detection, Analysis & Prediction Through Machine Learning**” being submitted by **Mirza Yasser Ali Baig (21WJ1A6642)**, **Syed Affan Ahmed (21WJ1A6653)** and **Mohammed Shahnawaz (22WJ5A6603)** in partial fulfillment for the award of the Degree of **Bachelor of Technology in Computer Science & Engineering (Artificial Intelligence and Machine Learning)** of the **Guru Nanak Institutions Technical Campus, Hyderabad** during the academic year 2024-2025, is a record of Bonafide work carried out under our guidance and supervision at **Guru Nanak Institutions Technical Campus (Autonomous)**.

Dr. S. Madhu
INTERNAL GUIDE

P. Venkata Pratima
PROJECT COORDINATOR

Dr. S. Madhu
HOD CSE(AIML)

EXTERNAL EXAMINER

Acknowledgement

We wish to express our sincere thanks to **Dr. Harvinder Singh Saini, Managing Director, GNITC** for providing us with the conducive environment for carrying through our academic schedules and project with ease.

We express our deepest gratitude to **Dr. S. Sreenatha Reddy, Director, GNITC**, for his constant support and encouragement and for providing us with all the facilities in the college during our project work.

We are thankful to **Dr. P. Parthasaradhy, Joint Director, GNITC** for his good support and given valuable inputs in our project work.

We wish to express our sincere thanks to **Dr. Rishi Sayal, Professor & Associate Director GNITC** for providing us with the conducive environment for carrying through our academic schedules and project with ease.

We sincerely thank **Dr. S. Madhu, Professor and HOD** of CSE (AIML), GNITC, Hyderabad, for his invaluable insights, guidance, and unwavering support throughout our project. His exceptional mentorship and technical expertise have been instrumental in the successful completion of our Bachelor's degree. His encouragement and contributions to both our dissertation and professional development have provided us with a rewarding and enriching experience, preparing us to achieve our future goals with confidence.

We are very much grateful to our **Project Coordinator, Mrs. P Venkata Pratima, Assistant Professor** Department of **CSE(AIML)**, GNITC, Hyderabad, who has not only shown utmost patience but were versatile in suggestions, vigilant in directions of error, and have been infinitely helpful.

On a more personal note, we thank our beloved parents and friends for their moral support during our project.

Mirza Yasser Ali Baig (21WJ1A6642)

Syed Affan Ahmed (21WJ1A6653)

Mohammed Shahnawaz (22WJ5A6603)

ABSTRACT:

Crime prediction and analysis play a pivotal role in enhancing public safety and streamlining law enforcement strategies, particularly in regions facing significant socio-economic challenges. This research presents a comprehensive framework leveraging machine learning techniques to analyze crime patterns, detect high-risk zones, and predict future criminal activities. By processing extensive datasets, including crime records and demographic information, the study identifies meaningful patterns to guide effective resource allocation. The proposed approach employs K-Means clustering to analyze spatial crime hotspots, enabling the identification of areas with a higher propensity for criminal activities. For temporal crime prediction, ARIMA and SARIMA models are utilized, offering robust time-series forecasting to identify trends and periods of heightened criminal activity. Random Forest and K-Nearest Neighbor (KNN) classifiers are incorporated to enhance classification accuracy and support suspect identification, efficiently categorizing crime types and analyzing criminal behavior. Dimensionality reduction through Principal Component Analysis (PCA) optimizes clustering by reducing data complexity while preserving key attributes, improving interpretability and visual clarity. This research demonstrates the transformative potential of advanced machine learning techniques in addressing real-world challenges in crime prevention. By providing precise, data-driven predictions and detailed visualizations, the system facilitates proactive decision-making, supports effective resource deployment, and contributes to safer, well-informed communities, underscoring the critical role of artificial intelligence in shaping the future of law enforcement and public safety.

TABLE OF CONTENT

CHAPTER NO.	TITLE	PAGE NO.
	ABSTRACT	i
	LIST OF FIGURES	v
	LIST OF SYMBOLS	vi
1.	CHAPTER 1 : INTRODUCTION	1-7
	1.1 INTRODUCTION	1
	1.2 SCOPE OF THE PROJECT	2
	1.3 OBJECTIVE	2
	1.4 EXISTING SYSTEM	3
	1.4.1 EXISTING SYSTEM DISADVANTAGES	3
	1.5 LITERATURE SURVEY	4-6
	1.6 PROPOSED SYSTEM	7
	1.6.1 PROPOSED SYSTEM ADVANTAGES	7
2.	CHAPTER 2 :PROJECT DESCRIPTION	8-10
	2.1 GENERAL	8
	2.2 METHODOLOGIES	9
	2.2.1 MODULES NAME	9
	2.2.2 MODULES EXPLANATION	9-10
	2.3 TECHNIQUE OR ALGORITHM	11
3.	CHAPTER 3 : REQUIREMENTS	12-14
	3.1 GENERAL	12
	3.2 HARDWARE REQUIREMENTS	12
	3.3 SOFTWARE REQUIREMENTS	13
	3.4 FUNCTIONAL REQUIREMENTS	13
	3.5 NON FUNCTIONAL REQUIREMENTS	14
4.	CHAPTER 4 :SYSTEM DESIGN	15-25
	4.1 GENERAL	15

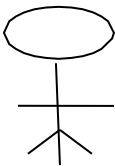
	4.2 UML DIAGRAMS	15
	4.2.1 USE CASE DIAGRAM	15
	4.2.2 CLASS DIAGRAM	16
	4.2.3 OBJECT DIAGRAM	17
	4.2.4 STATE DIAGRAM	18
	4.2.5 ACTIVITY DIAGRAM	19
	4.2.6 SEQUENCE DIAGRAM	20
	4.2.7 COLLABORATION DIAGRAM	21
	4.2.8 COMPONENT DIAGRAM	22
	4.2.9 DATA FLOW DIAGRAM	23
	4.2.10 DEPLOYMENT DIAGRAM	24
	4.2.11 SYSTEM ARCHITECTURE	25
5.	CHAPTER 5 :DEVELOPMENT TOOLS	26-28
	5.1 GENERAL	26
	5.2 HISTORY OF PYTHON	26
	5.3 IMPORTANCE OF PYTHON	26
	5.4 FEATURES OF PYTHON	27
	5.5 LIBRARIES USED IN PYTHON	28
6.	CHAPTER 6 :IMPLEMENTATION	29-51
	6.1 GENERAL	29
	6.2 IMPLEMENTATION	29-51
7.	CHAPTER 7 :SNAPSHOTS	52-53
	7.1 GENERAL	52
	7.2 SNAPSHOTS	53
8.	CHAPTER 8 :SOFTWARE TESTING	54-56
	8.1 GENERAL	54
	8.2 DEVELOPING METHODOLOGIES	54
	8.3 TYPES OF TESTING	54

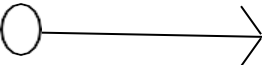
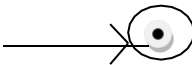
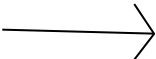
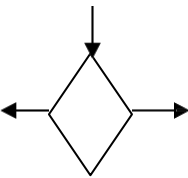
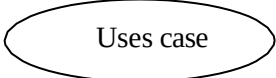
9.	CHAPTER 9 : CONCLUSION & REFERENCES	57- 58
	CONCLUSION	57
	REFERENCES	58

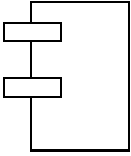
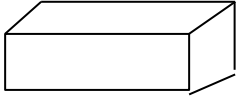
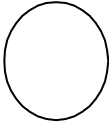
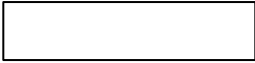
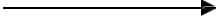
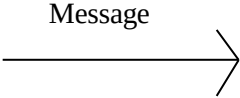
LIST OF FIGURES

FIGURE NO	NAME OF THE FIGURE	PAGE NO.
4.2.1	Use case Diagram	14
4.2.2	Class Diagram	15
4.2.3	Object Diagram	16
4.2.4	State Diagram	16
4.2.5	Activity Diagram	17
4.2.6	Sequence Diagram	18
4.2.7	Collaboration Diagram	19
4.2.8	Component Diagram	20
4.2.9	Data flow Diagram	21
4.2.10	Deployment Diagram	23
4.2.11	System Architecture Diagram	24

LIST OF SYMBOLS

S.NO	NOTATION NAME	NOTATION	DESCRIPTION
1.	Class	<i>+ public</i> <i>-private</i> <i>Class Name</i> <i>-attribute</i> <i>-attribute</i>	Represents a collection of similar entities grouped together.
2.	Association	Class A Class B Class A Class B	Associations represents static relationships between classes. Roles represents the way the two classes see each other.
3.	Actor		It aggregates several classes into a single classes.
4.	Aggregation	Class A Class B Class A Class B	Interaction between the system and external environment

5.	Relation (uses)	Uses	Used for additional process communication.
6.	Relation (extends)		Extends relationship is used when one use case is similar to another use case but does a bit more.
7.	Communication		Communication between various use cases.
8.	State		State of the processes.
9.	Initial State		Initial state of the object
10.	Final state		Final state of the object
11.	Control flow		Represents various control flow between the states.
12.	Decision box		Represents decision making process from a constraint
13.	Use case		Interaction between the System and external environment.

14.	Component		Represents physical modules which are a collection of nodes.
15.	Node		Represents physical modules which are a collection of components.
16.	Data Process/State		A circle in DFD represents a state or process which has been triggered due to some event or action.
17.	External entity		Represents external entities such as keyboard, sensors, etc.
18.	Transition		Represents communication that occurs between processes.
19.	Object Lifeline		Represents the vertical dimensions that the object communications.
20.	Message		Represents the message exchanged.

CHAPTER-1

INTRODUCTION

INTRODUCTION:

Crime poses a significant threat to societal well-being, governance, and economic stability. As cities grow and become increasingly interconnected, the complexity and frequency of crimes present significant challenges to law enforcement agencies. Traditional methods of crime analysis and prevention often rely on manual inspection and static reporting, which fail to provide actionable insights in real time. These methods struggle to process the vast amounts of data generated daily, leading to inefficiencies in identifying patterns and addressing criminal activities proactively. Consequently, there is an urgent need for advanced tools capable of analyzing and predicting crime patterns effectively.

Recent advancements in computational technologies, particularly in machine learning and data mining, offer a promising solution to these challenges. By analyzing historical crime data, machine learning models can uncover hidden patterns, forecast future crimes, and pinpoint hotspots where crimes are likely to occur. These insights empower law enforcement agencies to allocate resources efficiently and take preventive measures, thereby enhancing public safety. For instance, clustering techniques help identify geographical regions with high crime rates, while time series forecasting predicts when crimes are likely to increase, allowing for timely interventions.

This study adopts an interdisciplinary approach, combining machine learning, criminology, and data science to tackle crime analysis and prediction. Our project focuses on spatial and temporal analysis of crimes using real-world datasets. We utilize clustering algorithms like K-Means to identify crime hotspots and time series models such as ARIMA and SARIMA to forecast future crime trends. Furthermore, the integration of demographic data enriches the analysis, providing additional context to better understand the socio-economic factors contributing to crime. The results are visualized through intuitive graphs and heatmaps to enable informed decision-making by both law enforcement and the public.

This project not only highlights the potential of machine learning in crime analysis but also bridges the gap between computational science and criminology. By developing a robust framework for understanding and mitigating crime, we aim to make cities safer and foster a sense of security for communities worldwide.

1.1 SCOPE OF THE PROJECT

Our project, "Crime Prediction and Analysis," leverages advanced machine learning techniques to enhance public safety and support law enforcement in crime prevention. It focuses on identifying crime hotspots through spatial analysis, predicting future trends using temporal data, and classifying crimes by type for effective interventions. Algorithms such as KMeans, PCA, Random Forest, ARIMA, and SARIMA are employed to process and analyze crime data efficiently. The project provides actionable insights and visualizations, enabling better decision-making and resource allocation. With scalable and real-time capabilities, it facilitates proactive measures to address emerging crime patterns. By fostering data-driven solutions, our project contributes to predictive policing and aims to build safer communities through effective crime analysis and prevention strategies.

1.2 OBJECTIVE

The objective of this project is to develop a machine learning-based framework that enhances public safety and supports proactive law enforcement strategies by analyzing and predicting crime patterns. The system aims to identify high-risk areas or crime hotspots using clustering techniques and predict future crimes through time-series forecasting models. It further seeks to classify crimes based on key attributes such as type, location, and time of occurrence, utilizing advanced classification algorithms. By providing actionable insights, the system enables law enforcement agencies to allocate resources more effectively and efficiently, ensuring targeted interventions in areas of concern. Additionally, the project emphasizes the use of data-driven visualizations to support informed decision-making and planning for crime prevention. Ultimately, this framework aspires to transform traditional, reactive approaches to crime analysis into a proactive, AI-driven methodology that prioritizes safety and resource optimization.

1.3 EXISTING SYSTEM:

Traditional crime analysis systems rely heavily on historical data and basic statistical methods to identify patterns and trends. These systems often use manual crime mapping and rule-based approaches to analyze crime incidents. While useful to some extent, they lack advanced predictive capabilities and fail to address the dynamic nature of crime. Existing systems primarily focus on reactive measures, responding to crimes after they occur rather than preventing them. Additionally, they struggle to process large datasets and visualize complex crime patterns, making them inefficient for modern crime

analysis.

1.3.1 EXISTING SYSTEM DISADVANTAGES:

- **Reactive Policing:** Focuses on responding to crimes rather than preventing them.
- **Limited Predictive Capabilities:** Cannot forecast future crimes or identify trends effectively.
- **Static Analysis:** Relies on historical data without accounting for dynamic factors like population growth or urban development.
- **Manual Effort:** Requires significant time and manpower, making the system inefficient and prone to errors.
- **Poor Visualization:** Lacks tools for understanding complex crime patterns and hotspots.
- **Inefficient Resource Allocation:** Results in suboptimal deployment of law enforcement resources.
- **Inaccurate Insights:** Absence of advanced algorithms leads to less reliable results. struggle with understanding relationships or broader contextual information within an image.

1.4 LITERATURE SURVEY

Title: Crime Data Visualization and Forecasting Using SARIMA Algorithm

Author : Safiya Mohd, Hithasri Muthyala

Year: 2023

Description: This research focuses on the application of the Seasonal Autoregressive Integrated Moving Average (SARIMA) algorithm for the visualization and forecasting of crime data in India. The study emphasizes the significance of analyzing historical crime data to identify patterns and trends that can inform law enforcement strategies. By employing SARIMA, the authors demonstrate how seasonal and trend components can be effectively modeled to predict future crime occurrences, thereby enhancing the ability of police and government agencies to allocate resources efficiently.

The research utilizes a comprehensive dataset that includes year-wise crime incidents across various states and districts in India. Through graphical representations such as heat maps, trend lines, and column charts, the study provides a clear visualization of crime rates, making it easier to identify hotspots and fluctuations over time. The findings reveal that the SARIMA model yields accurate forecasts, with performance metrics such as Mean Absolute Percentage Error (MAPE) indicating its reliability.

Moreover, the study highlights the importance of data normalization and the selection of optimal parameters for the SARIMA model to ensure robust predictions. By integrating these analytical techniques, the research aims to support law enforcement agencies in their efforts to reduce crime rates and improve public safety. Ultimately, the study advocates for the adoption of data-driven methodologies in crime analysis, underscoring their potential to transform how authorities approach crime prevention and intervention strategies.

Title: 'Big Data Analytics for Forecasting of Crime Data'

Author: Anusha K, Dr. M.K. Jayanti Kannan.

Year: 2020.

Description: The research focuses on the application of Big Data Analytics in forecasting crime data, specifically analyzing crime incidents in Vancouver over a period from 2003 to 2017. The study utilizes a dataset containing 530,652 records, which were collected from various sources, including social media and public databases. The methodology involves several steps, including data collection, preprocessing, and the implementation of machine learning algorithms to predict crime occurrences. The authors emphasize the importance of incorporating real-time data, such as social media activity, to capture short-term variations in crime patterns that traditional methods may overlook. Various techniques, including choropleth mapping, are employed to visualize crime hotspots, aiding law enforcement in operational decision-making. The research also addresses challenges such as missing values and outliers in the dataset, ensuring the robustness of the predictive models. By leveraging advanced analytical tools, the study aims to provide actionable insights that can enhance public safety and improve resource allocation for police departments. The findings highlight the potential of data-driven approaches in understanding criminal behavior and developing effective strategies for crime prevention and intervention.

Title: 'Proactive Spatiotemporal Resource Allocation and Predictive Visual Analytics for Community Policing and Law Enforcement'

Author: Abish Malik, Ross Maciejewski, Sherry Towers, Sean McCullough, David S. Ebert

Year:2014.

Description: This paper presents a method for simplifying temporal event sequences to enhance the visualization and analysis of complex time-oriented data. The authors propose a framework that allows users to interactively explore and manipulate event sequences, making it easier to identify patterns and trends over time. By employing techniques such as event clustering and abstraction, the method aims to reduce visual clutter while preserving essential information, thereby improving the overall effectiveness of visual analytics in various applications, including crime analysis and public health monitoring. The study demonstrates the utility of the approach through case studies, highlighting its potential to facilitate better decision-making in time-sensitive contexts.

Title: Big Data Analytics and Mining for Effective Visualization and Trends Forecasting of Crime Data.

Author: Mingchen Feng, Jiangbin Zheng, Jinchang Ren, Amir Hussain, Xiuxiu Li, Yue Xi, Qiaoyuan Liu.

Year:2019.

Description: The paper titled "Big Data Analytics and Mining for Effective Visualization and Trends Forecasting of Crime Data" explores the application of Big Data Analytics (BDA) in analyzing crime data from three major US cities: San Francisco, Chicago, and Philadelphia. The authors utilize state-of-the-art data mining and deep learning techniques to conduct exploratory data analysis, aiming to identify patterns and predict trends in criminal activities. The study highlights the importance of BDA in enhancing the understanding of crime issues, enabling law enforcement agencies to make informed decisions regarding resource allocation and incident prediction.

The research employs various advanced models, including the Prophet model and Keras stateful Long Short-Term Memory (LSTM) networks, to analyze the data. The findings indicate that both the Prophet model and LSTM outperform conventional neural network models in terms of predictive accuracy, with an optimal training period of three years identified for achieving the best results. The paper emphasizes the significance of extracting meaningful insights from large datasets, which can ultimately improve safety, security, and quality of life in urban environments.

In addition to presenting experimental results, the authors propose a novel visual representation technique that allows users to explore and analyze crime trends effectively. The paper is structured to include a literature review, detailed methodologies for data processing and visualization, and a discussion of the experimental outcomes. The authors also outline future plans to enhance their platform for generic big data analytics, incorporating multivariate visualization and graph mining techniques to uncover further patterns and trends. Overall, this research contributes valuable insights into the application of BDA in crime analysis, offering practical implications for law enforcement agencies

1.5 PROPOSED SYSTEM

Our proposed system utilizes advanced machine learning techniques to analyze and predict crime patterns effectively. K-Means Clustering is used to identify crime hotspots by grouping incidents based on geographic coordinates. By partitioning data into clusters, this method helps pinpoint high-crime areas, allowing law enforcement to focus resources strategically. Principal Component Analysis (PCA) simplifies complex crime datasets by reducing dimensionality, making it easier to visualize and interpret clustering results without losing critical information. To predict future crime trends, the system employs ARIMA (AutoRegressive Integrated Moving Average), a time-series forecasting model that uses historical data to anticipate crime occurrences. Additionally, SARIMA (Seasonal ARIMA) extends ARIMA to account for seasonal variations, such as recurring crime patterns during specific months or holidays, offering more accurate forecasts. These techniques together enable our system to provide actionable insights, such as identifying high-risk areas, understanding crime trends, and predicting potential future crimes. By leveraging these methodologies, our system supports law enforcement in planning and resource allocation, enhancing their ability to combat crime proactively and efficiently. The proposed system aims to strengthen public safety by combining spatial and temporal analysis for comprehensive crime prediction and prevention strategies.

1.5.1 PROPOSED SYSTEM ADVANTAGES:

- **Enhanced Predictive Accuracy:** Machine learning algorithms like K-Means, PCA, Random Forest, ARIMA, and SARIMA handle complex data and provide accurate predictions of crime locations and times.
- **Real-Time Decision Making:** Enables dynamic, data-driven decisions for patrol allocation, surveillance, and community-based prevention, addressing emerging crime trends effectively.
- **Spatial and Temporal Insights:** Identifies crime hotspots and forecasts future trends, allowing proactive planning and targeted interventions.
- **Improved Resource Allocation:** Optimizes the deployment of officers, surveillance, and awareness campaigns, ensuring resources are used efficiently.
- **Scalability and Adaptability:** The system evolves with new data, remains accurate over time, and can be applied across different regions or contexts.

CHAPTER 2

PROJECT DESCRIPTION

2.1 GENERAL:

Using a comprehensive crime dataset, this project aims to develop a reliable machine learning-based system for detecting, analyzing, and predicting crime patterns in urban areas. The implementation of various methods, including Random Forest, K-Nearest Neighbors (KNN), and Logistic Regression, addresses challenges such as high-class imbalance within the dataset. The dataset is divided into training and testing subsets to evaluate the models, and feature selection techniques are applied to enhance their performance.

The study evaluates each algorithm's ability to identify spatial and temporal crime hotspots, forecast potential crime occurrences, and predict crime types based on factors like occurrence date, reporting date, neighborhood, and type of offense. Performance is measured not only by accuracy but also through metrics like precision, recall, F1-score, and AUC-ROC, ensuring a thorough assessment.

The results indicate that the proposed system effectively identifies and predicts crime patterns. It also holds significant potential for practical applications, such as highlighting high-risk neighborhoods for law enforcement focus, improving public safety awareness, and supporting proactive policing strategies. This comprehensive approach combines crime dataset findings with demographic information, offering deeper insights into urban crime dynamics.

2.2 METHODOLOGIES

2.2.1 MODULES NAME:

Modules Name:

- Data Gathering
- Classification / Clustering
- Prediction / Forecasting
- Exploratory Data Analysis

2.2.2 MODULES EXPLANATION:

1. Data Gathering:

Data gathering is the most critical and initial step of the Crime Prediction and Analysis project, as it forms the foundation for all subsequent processes. It involves the systematic collection of raw crime data from credible sources such as Kaggle, government crime records, or law enforcement databases. The dataset typically includes essential details like crime type, date, time, location, and demographic information, ensuring a comprehensive understanding of the crime scenario. Data is usually stored in structured formats like CSV files, making it easier to clean, preprocess, and analyze. Ensuring data accuracy, completeness, and diversity at this stage is vital, as the quality of collected data significantly influences the reliability of the entire analysis pipeline. Proper data collection enables the project to address a wide variety of crime-related questions and ensures that the models developed have sufficient input to perform robust predictions.

2. Classification / Clustering:

Classification and clustering are pivotal for organizing and understanding the crime data. Classification, implemented through Random Forest, helps in categorizing crimes into specific types like assault, robbery, and burglary. The Random Forest algorithm works by constructing multiple decision trees during training and voting for the most accurate output, ensuring high accuracy in predictions. On the other hand, KMeans clustering, an unsupervised learning technique, is used to

group similar geographic data points to identify crime hotspots. By analyzing longitude and latitude values, KMeans can effectively detect high-crime areas, enabling law enforcement to allocate their resources to regions with the most urgent needs. These methods not only improve crime categorization but also provide actionable insights into spatial crime patterns, helping in crime prevention and resource optimization.

3. Prediction / Forecasting:

Prediction and forecasting play a crucial role in preparing law enforcement agencies for future crime trends. This module uses ARIMA (Auto-Regressive Integrated Moving Average) to analyze historical crime data and forecast future crime rates. ARIMA identifies patterns, such as increases or decreases in crime frequency, and predicts trends over a specific time horizon. To enhance prediction accuracy, SARIMA (Seasonal ARIMA) is also employed, incorporating seasonal variations like holidays, weekends, or festival periods when crime rates often surge. By accounting for such patterns, SARIMA significantly improves the precision of the forecasts. These predictions provide actionable insights into when and where crimes are likely to occur, allowing law enforcement agencies to plan preventive measures, deploy resources effectively, and mitigate risks before they escalate.

4. Exploratory Data Analysis:

Exploratory Data Analysis (EDA) is an essential step for understanding the dataset's structure and uncovering meaningful insights. This module involves summarizing and visualizing the dataset using techniques like heatmaps, scatter plots, and bar charts to highlight patterns and anomalies. For instance, heatmaps can reveal areas with high crime density, while scatter plots and temporal graphs identify peak crime times or days. Statistical analyses, such as mean, median, and variance, help in understanding the distribution of data attributes, while correlation matrices explore relationships between variables like crime type and location. By identifying key trends, anomalies, and relationships, EDA enables better-informed decisions for model building and helps law enforcement agencies prioritize their efforts in high-crime areas and periods. This step is critical for creating a reliable and actionable predictive system.

2.3 TECHNIQUE USED AND ALGORITHM USED

This project employs advanced machine learning techniques and algorithms to achieve its objectives of crime hotspot detection, trend forecasting, and classification. The techniques used include clustering, classification, dimensionality reduction, and time series analysis. KMeans Clustering, an unsupervised learning algorithm, is utilized to identify spatial crime hotspots by grouping incidents based on geographic coordinates. This technique effectively detects high-crime areas, enabling law enforcement to focus their efforts on these zones. To simplify complex datasets, Principal Component Analysis (PCA) is employed as a dimensionality reduction technique. PCA reduces the dataset's dimensions while retaining the most significant features, making it easier to visualize and interpret results without losing critical information.

For crime type classification, the project uses the Random Forest algorithm, a powerful supervised learning method. Random Forest builds multiple decision trees during training and combines their outputs to enhance classification accuracy and robustness. This approach helps categorize crimes into types such as theft, assault, and robbery, providing deeper insights into crime patterns.

To predict future crime trends, the project integrates time series forecasting models like ARIMA (Auto-Regressive Integrated Moving Average) and SARIMA (Seasonal ARIMA). ARIMA is used to model historical crime data and predict future crime rates by identifying trends and patterns in temporal data. SARIMA, an extension of ARIMA, incorporates seasonal variations such as recurring crime surges during holidays or weekends, improving the model's accuracy in forecasting cyclical crime trends.

These algorithms work in unison to provide a comprehensive analysis of crime data. While KMeans and PCA focus on spatial analysis, Random Forest addresses categorical insights, and ARIMA/SARIMA handle temporal patterns. Together, these techniques ensure the system delivers actionable insights, aiding law enforcement in making informed decisions, allocating resources strategically, and implementing effective crime prevention measures.

CHAPTER 3

REQUIREMENTS ENGINEERING

3.1 GENERAL

We can see from the results that on each database, the error rates are very low due to the discriminatory power of features and the regression capabilities of classifiers. Comparing the highest accuracies (corresponding to the lowest error rates) to those of previous works, our results are very competitive.

3.2 HARDWARE REQUIREMENTS

The hardware requirements may serve as the basis for a contract for the implementation of the system and should therefore be a complete and consistent specification of the whole system. They are used by software engineers as the starting point for the system design. It should what the system do and not how it should be implemented.

- PROCESSOR : Pentium i3 Processor.
- RAM : 4GB DD RAM
- HARD DISK : 250 GB

3.3 SOFTWARE REQUIREMENTS

The software requirements document is the specification of the system. It should include both a definition and a specification of requirements. It is a set of what the system should do rather than how it should do it. The software requirements provide a basis for creating the software requirements specification. It is useful in estimating cost, planning team activities, performing tasks and tracking the teams and tracking the team's progress throughout the development activity.

- Operating System : Windows 7/8/10
- Platform : Anaconda
- Programming Language : Python
- Front End : Spyder3

3.4 FUNCTIONAL REQUIREMENTS

A functional requirement defines a function of a software-system or its component. A function is described as a set of inputs, the behavior. Firstly, the system is the first that achieves the standard notion of semantic security for data confidentiality in attribute-based deduplication systems by resorting to the hybrid cloud architecture.

3.5 NON-FUNCTIONAL REQUIREMENTS

The major non-functional Requirements of the system are as follows

- **Usability**

The system is designed with completely automated process hence there is no or less user intervention.

- **Reliability**

The system is more reliable because of the qualities that are inherited from the chosen platform python. The code built by using python is more reliable.

- **Performance**

This system is developing in the high level languages and using the advanced back-end technologies it will give response to the end user on client system with in very less time.

- **Supportability**

The system is designed to be the cross platform supportable. The system is supported on a wide range of hardware and any software platform, which is built into the system.

- **Implementation**

The system is implemented in web environment using Jupyter notebook software. The server is used as the intelligence server and windows 10 professional is used as the platform. Interface the user interface is based on Jupyter notebook provides server system.

CHAPTER 4 DESIGN ENGINEERING

4.1 GENERAL

Design Engineering deals with the various UML [Unified Modelling language] diagrams for the implementation of project. Design is a meaningful engineering representation of a thing that is to be built. Software design is a process through which the requirements are translated into representation of the software. Design is the place where quality is rendered in software engineering.

4.2 UML DIAGRAMS

4.2.1 USE CASE DIAGRAM

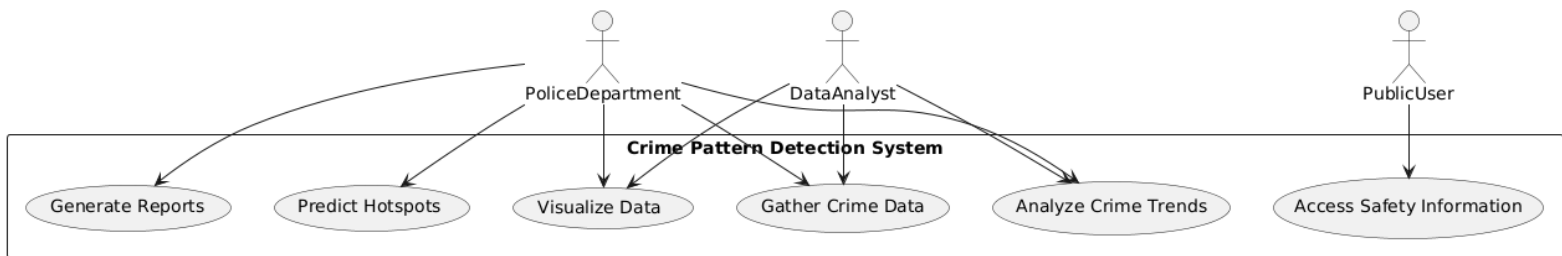


Fig 4.2.1: Use Case Diagram

EXPLANATION:

The Use Case Diagram illustrates the interactions between various actors and the Crime Pattern Detection System, highlighting the functionalities available to each user. It identifies three primary actors—Police Department, Data Analyst, and Public User—and outlines key use cases such as gathering crime data, analyzing trends, predicting hotspots, and accessing safety information. This diagram serves to clarify system requirements and user roles within the crime analysis framework.

4.2.2 CLASS DIAGRAM

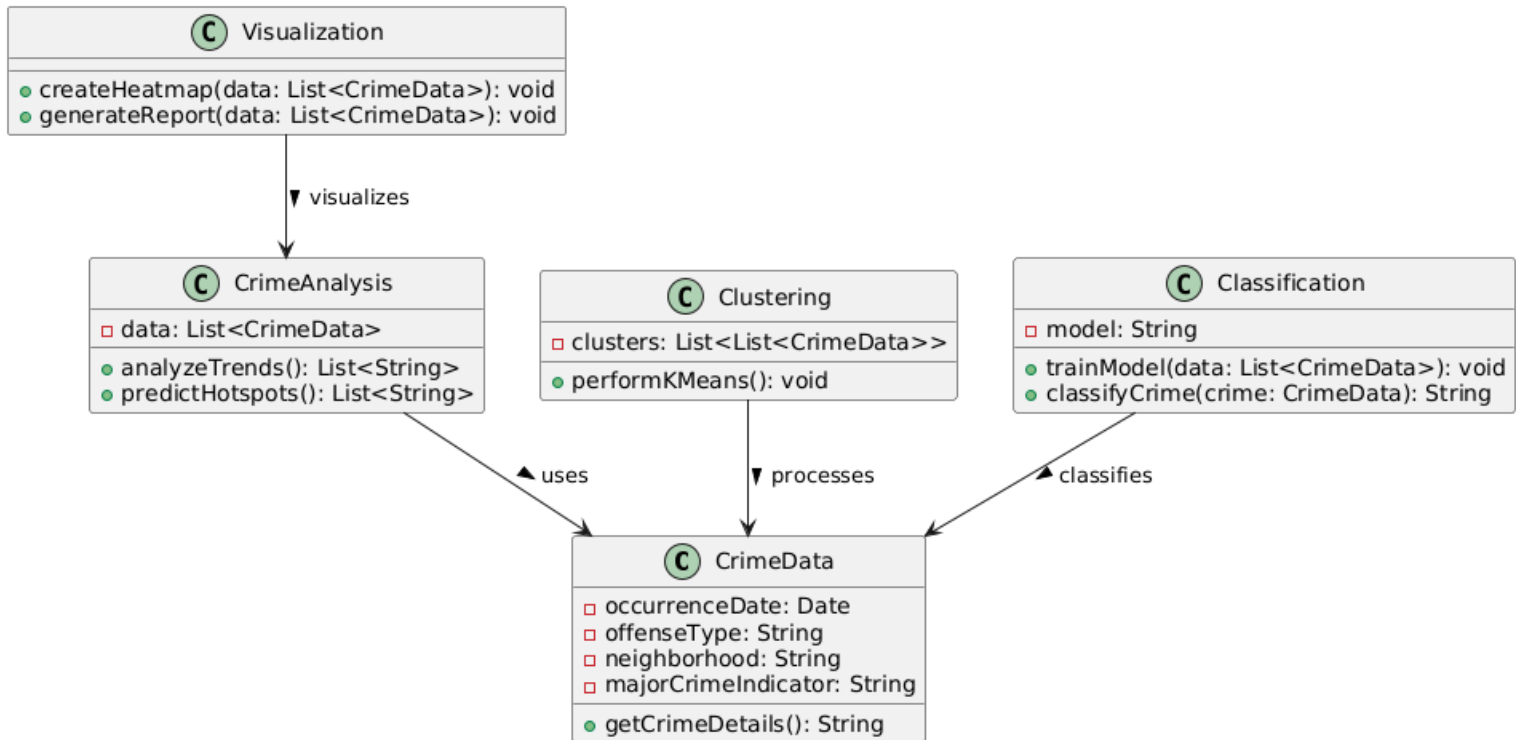


Fig 4.2.2: Class Diagram

EXPLANATION

The Class Diagram for the crime pattern detection system outlines the system's architecture by defining key classes that represent various components of the system. It includes classes like CrimeData, which encapsulates attributes such as occurrence date, offense type, and neighborhood, along with methods for retrieving crime details. The CrimeAnalysis class utilizes collections of CrimeData to analyze trends and predict hotspots, while the Clustering and Classification classes implement algorithms for grouping and categorizing crime data. The Visualization class is responsible for presenting the analysis results visually. This diagram illustrates the relationships between these classes, highlighting how they interact to facilitate crime analysis and prediction.

4.2.3 OBJECT DIAGRAM

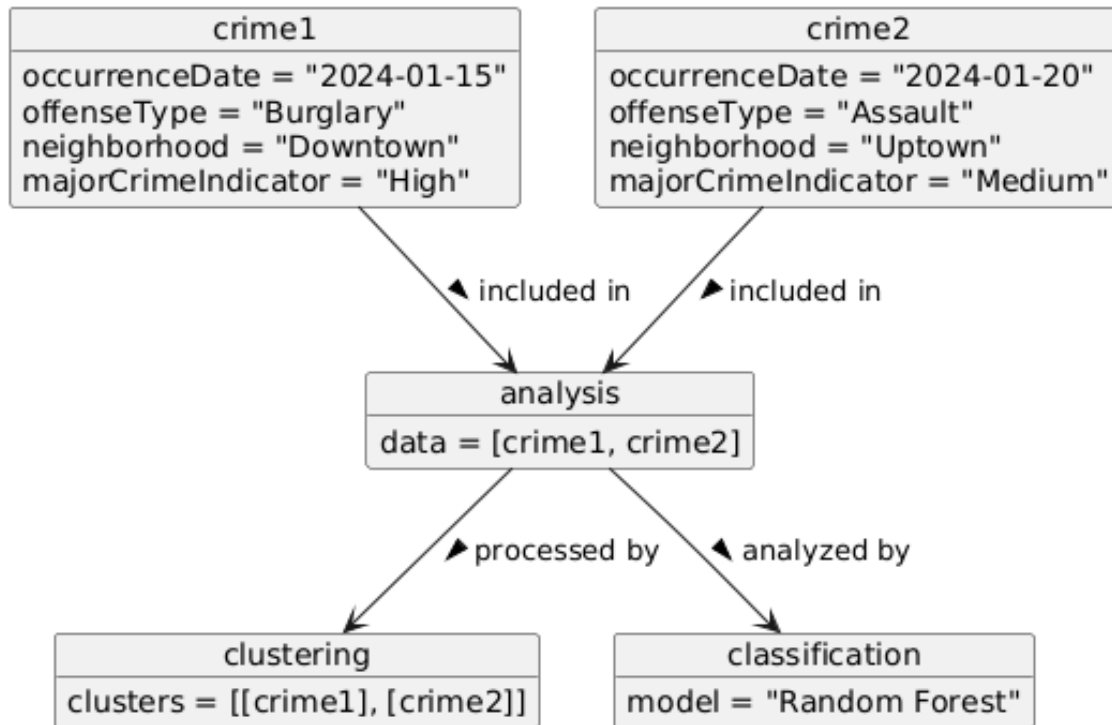


Fig 4.2.3: Object Diagram

EXPLANATION:

The Object Diagram provides a snapshot of specific instances of the classes defined in the class diagram at a particular moment in time. It includes instances such as **crime1** and **crime2**, representing individual crime records with specific attributes like occurrence date and offense type. The diagram also shows how these instances are aggregated within an **analysis** object, which collects multiple **CrimeData** instances for analysis. Additionally, it depicts how the **clustering** object groups these crimes into clusters based on their characteristics, while the **classification** object indicates the machine learning model being used for categorization. This diagram emphasizes the dynamic interactions between objects in the system, showcasing how individual crime incidents are processed collectively for insights and predictions.

4.2.4: STATE DIAGRAM

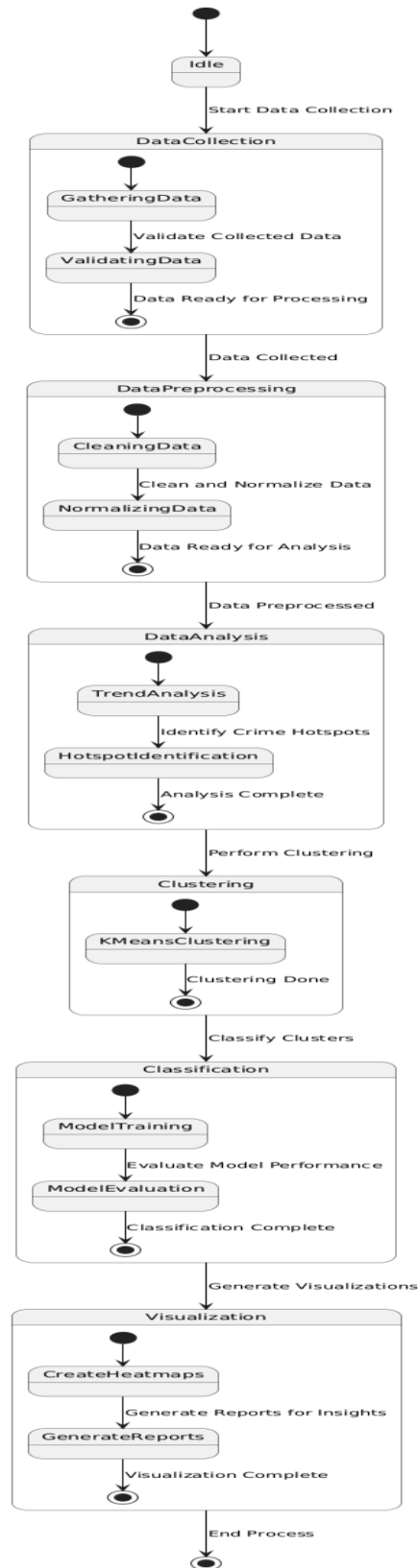


Fig 4.2.4: State Diagram

EXPLANATION:

The State Diagram for the crime pattern detection system outlines the various states the system undergoes during its operation, from data collection to visualization. It begins in an idle state and transitions through several key phases: collecting data, preprocessing it, analyzing trends, performing clustering and classification, and finally visualizing the results. Each state encompasses specific sub-states that detail the processes involved, such as gathering and validating data or training and evaluating models. This diagram effectively illustrates the workflow of the system, highlighting how it processes crime data to generate actionable insights for law enforcement and public safety.

4.2.5 ACTIVITY DIAGRAM

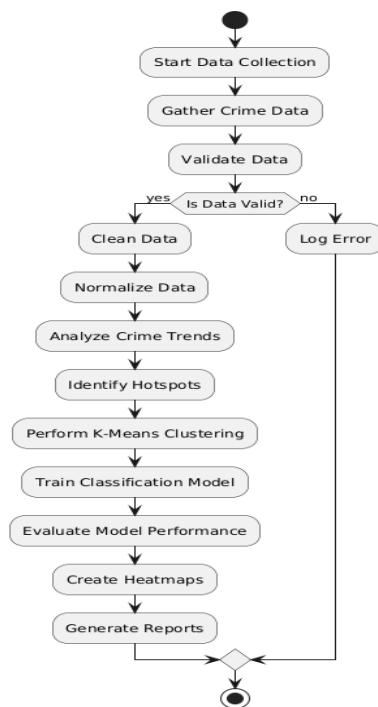


Fig 4.2.5: Activity Diagram

EXPLANATION:

This Activity Diagram depicts the workflow of the crime pattern detection system, illustrating the sequence of activities from data collection to report generation. It begins with the user initiating data

collection and gathering crime data, followed by a validation step. If the data is valid, it proceeds to cleaning, normalization, analysis of trends, hotspot identification, clustering, model training, and visualization. If the data is invalid, an error is logged. This diagram effectively captures the overall process flow and decision points within the system.

4.2.6 SEQUENCE DIAGRAM

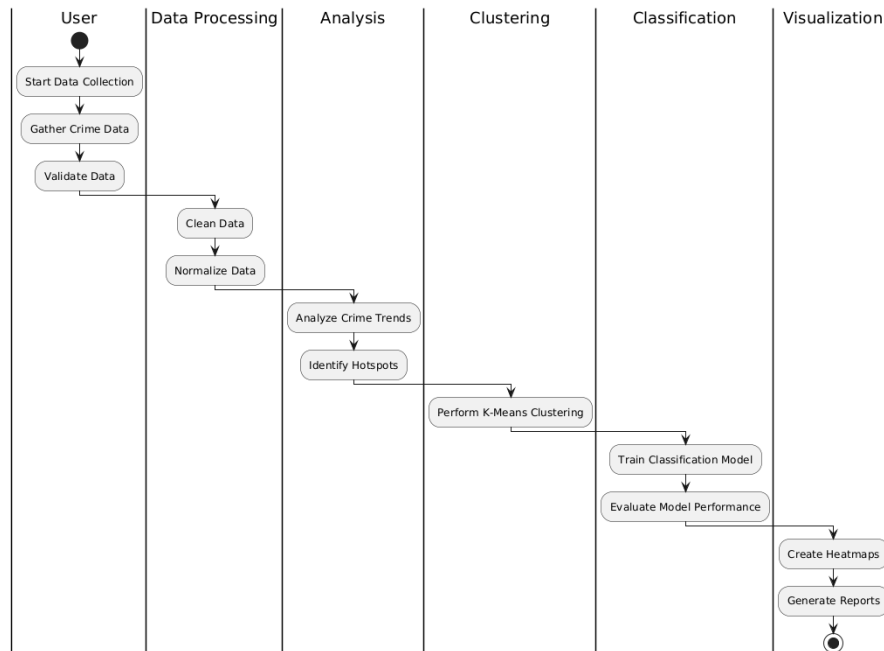


Fig 4.2.6: Sequence Diagram

EXPLANATION:

This Sequence Diagram outlines the workflow of the crime pattern detection system, illustrating the sequence of activities involved in processing crime data. It begins with the user initiating data collection, followed by data gathering and validation. The process then moves into data processing, where cleaning and normalization occur. Next, the system analyzes crime trends and identifies hotspots before performing clustering through K-Means. After clustering, a classification model is trained and evaluated. Finally, the results are visualized through heatmaps and reports, completing the workflow..

4.2.7 COLLABORATION DIAGRAM

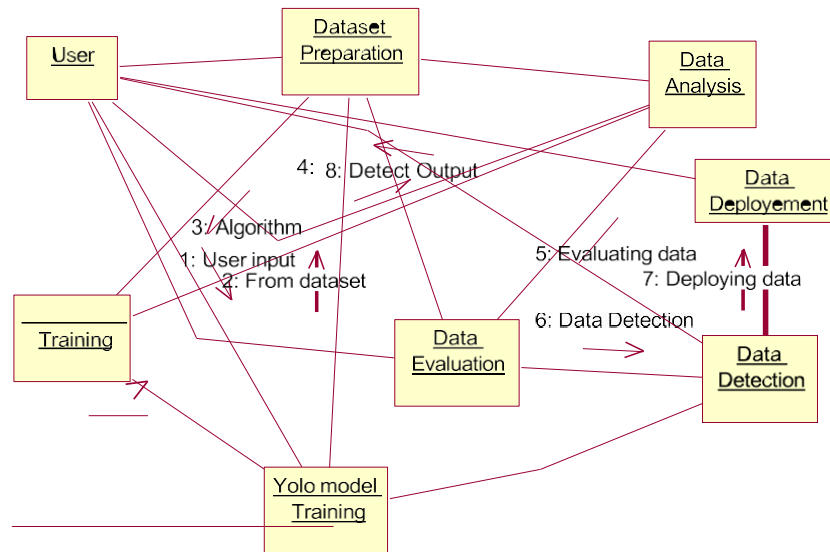


Fig 4.2.7: Collaboration Diagram

EXPLANATION:

A collaboration diagram, also called a communication diagram or interaction diagram, is an illustration of the relationships and interactions among software objects in the Unified Modeling Language (UML). The concept is more than a decade old although it has been refined as modeling paradigms have evolved.

4.2.8 COMPONENT DIAGRAM

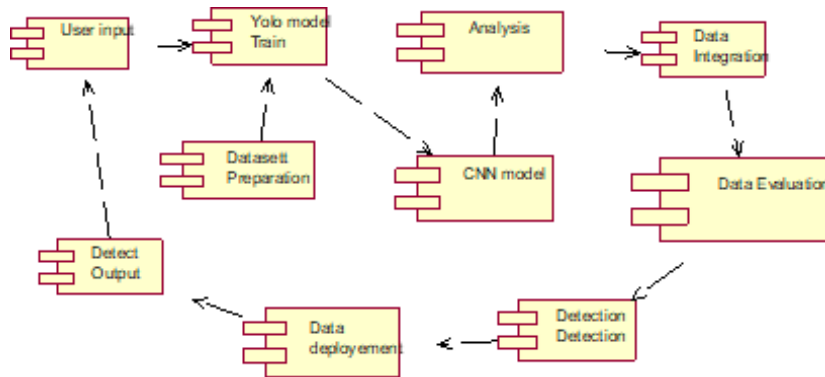


Fig 4.2.8: Component Diagram

EXPLANATION:

In the Unified Modeling Language, a component diagram depicts how components are wired together to form larger components and or software systems. They are used to illustrate the structure of arbitrarily complex systems. User gives main query and it converted into sub queries and sends through data dissemination to data aggregators. Results are to be showed to user by data aggregators. All boxes are components and arrow indicates dependencies.

4.2.9 Data Flow Diagram

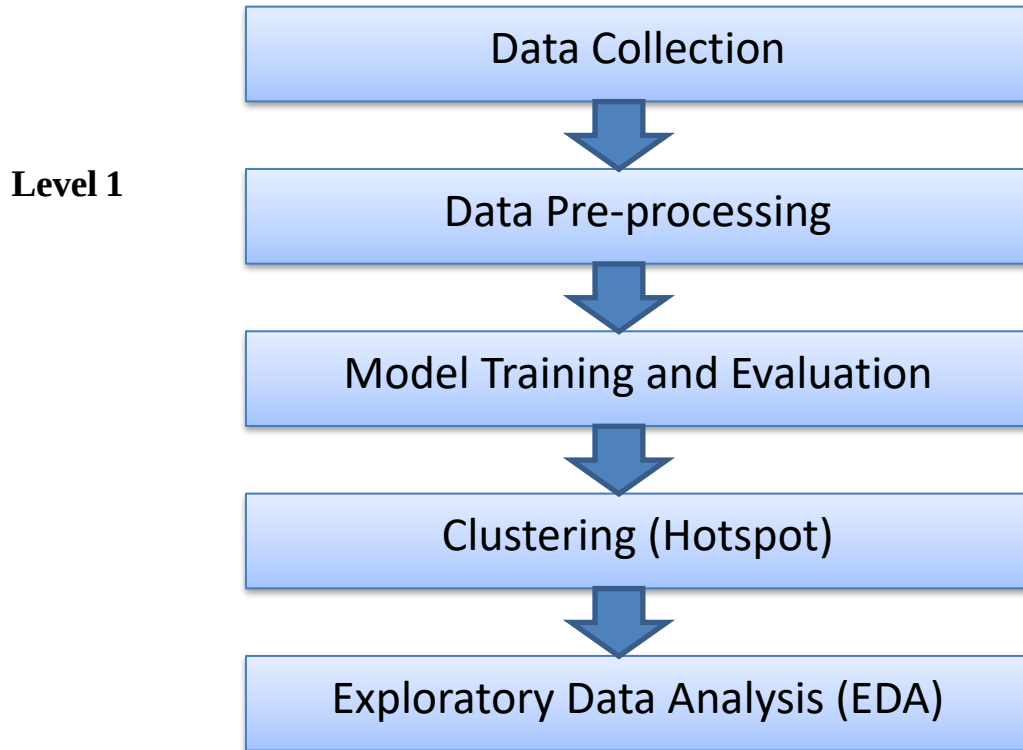


Fig 4.2.9.2: Data Flow Diagram

EXPLANATION:

A data flow diagram (DFD) is a graphical representation of the "flow" of data through an information system, modeling its process aspects. Often they are a preliminary step used to create an overview of the system which can later be elaborated. DFDs can also be used for the visualization of data processing (structured design).

A DFD shows what kinds of data will be input to and output from the system, where the data will come from and go to, and where the data will be stored. It does not show information about the timing of processes, or information about whether processes will operate in sequence or in parallel.

4.2.10 DEPLOYMENT DIAGRAM

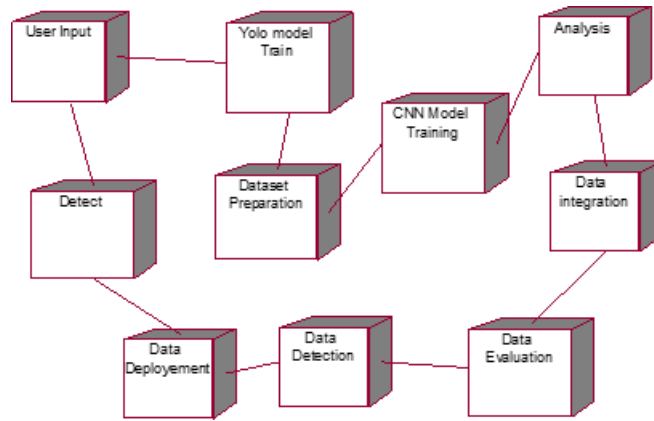


Fig 4.2.10: Deployment Diagram

EXPLANATION:

Deployment Diagram is a type of diagram that specifies the physical hardware on which the software system will execute. It also determines how the software is deployed on the underlying hardware. It maps software pieces of a system to the device that are going to execute it.

4.2.11. SYSTEM ARCHITECTURE:

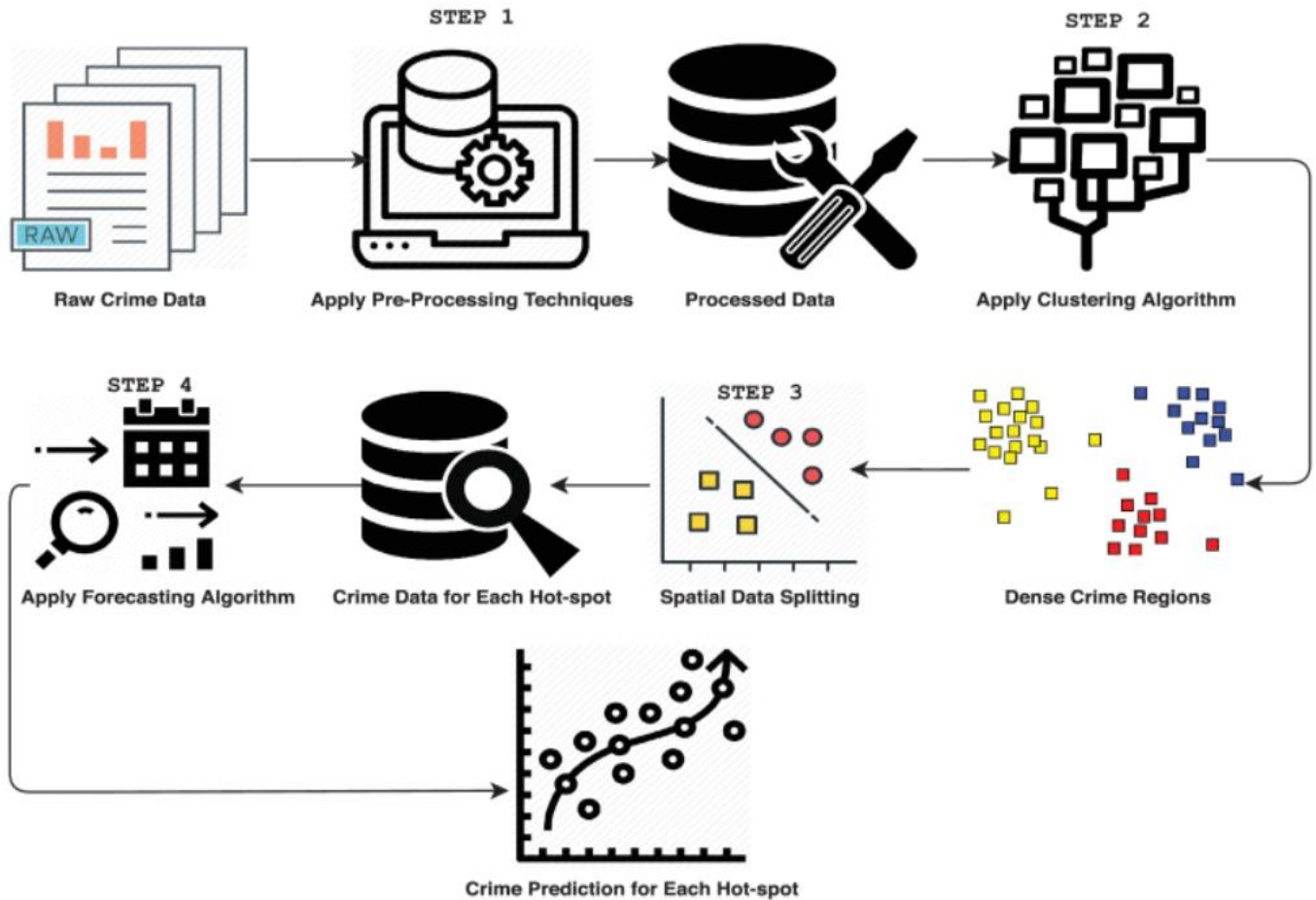


Fig 4.2.11: System Architecture

The system architecture for the proposed crime prediction and analysis model is designed to process large volumes of crime data and derive actionable insights. It begins with a Data Collection Layer, gathering historical crime data and relevant contextual factors like demographics and socioeconomic details. The Preprocessing Layer cleans and prepares the data by handling missing values, normalizing features, and encoding categorical variables. The Model Training Layer applies machine learning algorithms such as K-Means for clustering, ARIMA/SARIMA for time-series forecasting, and Random Forest for classification. A Prediction Layer generates insights like crime hotspots, future crime trends, and classifications, while the Visualization and Reporting Layer presents these results through intuitive dashboards, heatmaps, and trend graphs. This modular architecture ensures scalability, adaptability, and real-time applicability, making it a robust tool for proactive crime prevention and resource optimization.s.

CHAPTER 5

DEVELOPMENT TOOLS

5.1 Python

Python is a high-level, interpreted, interactive and object-oriented scripting language. Python is designed to be highly readable. It uses English keywords frequently where as other languages use punctuation, and it has fewer syntactical constructions than other languages.

5.2 History of Python

Python was developed by Guido van Rossum in the late eighties and early nineties at the National Research Institute for Mathematics and Computer Science in the Netherlands. Python is derived from many other languages, including ABC, Modula-3, C, C++, Algol- 68, Small Talk, and Unix shell and other scripting languages. Python is copyrighted. Like Perl, Python source code is now available under the GNU General Public License (GPL). Python is now maintained by a core development team at the institute, although Guido van Rossum still holds a vital role in directing its progress.

5.3 Importance of Python

- **Python is Interpreted** – Python is processed at runtime by the interpreter. You do not need to compile your program before executing it. This is similar to PERL and PHP.
- **Python is Interactive** – You can actually sit at a Python prompt and interact with the interpreter directly to write your programs.
- **Python is Object-Oriented** – Python supports Object-Oriented style or technique of programming that encapsulates code within objects.
- **Python is a Beginner's Language** – Python is a great language for the beginner-level programmers and supports the development of a wide range of applications from simple text processing to WWW browsers to games.

5.4 Features of Python

- **Easy-to-learn** – Python has few keywords, simple structure, and a clearly defined syntax. This allows the student to pick up the language quickly.
- **Easy-to-read** – Python code is more clearly defined and visible to the eyes.
- **Easy-to-maintain** – Python's source code is fairly easy-to-maintain.
- **A broad standard library** – Python's bulk of the library is very portable and cross-platform compatible on UNIX, Windows, and Macintosh.
- **Interactive Mode** – Python has support for an interactive mode which allows interactive testing and debugging of snippets of code.
- **Portable** – Python can run on a wide variety of hardware platforms and has the same interface on all platforms.
- **Extendable** – You can add low-level modules to the Python interpreter. These modules enable programmers to add to or customize their tools to be more efficient.
- **Databases** – Python provides interfaces to all major commercial databases.
- **GUI Programming** – Python supports GUI applications that can be created and ported to many system calls, libraries and windows systems, such as Windows MFC, Macintosh, and the X Window system of Unix.
- **Scalable** – Python provides a better structure and support for large programs than shell scripting.

Apart from the above-mentioned features, Python has a big list of good features, few are listed below –

- It supports functional and structured programming methods as well as OOP.
- It can be used as a scripting language or can be compiled to byte-code for building large applications.
- It provides very high-level dynamic data types and supports dynamic type checking.
- IT supports automatic garbage collection.
- It can be easily integrated with C, C++, COM, ActiveX, CORBA, and Java.

5.5 Libraries used in python

- numpy - mainly useful for its N-dimensional array objects.
- pandas - Python data analysis library, including structures such as dataframes.
- matplotlib - 2D plotting library producing publication quality figures.
- scikit-learn - the machine learning algorithms used for data analysis and data mining tasks.

CHAPTER 6

IMPLEMENTATION

6.1 GENERAL

The Implementation is nothing but source code of project.

Coding:

clustering.ipynb:

```
import numpy as np
import timeit
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.cluster import KMeans
import matplotlib.pyplot as plt
import matplotlib
from sklearn.decomposition import PCA
from sklearn.metrics import silhouette_samples, silhouette_score
import matplotlib.cm as cm
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import OneHotEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.ensemble import GradientBoostingClassifier
from sklearn.metrics import confusion_matrix, classification_report, accuracy_score

df=pd.read_csv("C:\Users\91770\Downloads\MCI_2014_to_2019.csv")
df["Total"] = 1
df.head()

df.dtypes
```



```

df.dropna()
print('Original Data Size after dropping Duplicates')
df = df.drop_duplicates(subset='event_unique_id',keep='first')
df.shape

drop_colmns = ['X', 'Y', 'Index_', 'reporteddate', 'reportedyear', 'reportedmonth', 'reportedday',
'reporteddayofyear',
                'reporteddayofweek', 'reportedhour', 'Hood_ID', 'ucr_code', 'ucr_ext', 'Division',
'occurrencedayofyear']
df_dropped = df.drop(columns=drop_colmns)

df_dropped.dtypes

assault = df[df['MCI'] == 'Assault']
assault_types = assault.groupby('offence',as_index=False).size()
print(assault_types)
ct = assault_types.sort_values(ascending = False)
ax = ct.plot.bar()
ax.set_xlabel('Types of Assault')
ax.set_ylabel('Number of occurences')
ax.set_title('Assault crimes in Toronto',color = 'green',fontsize=20)
plt.show()

df_grouped = df_dropped.groupby(df_dropped['occurrenceyear'])

#Analysis by year
df_2015 = df_grouped.get_group(2015)
df_2016 = df_grouped.get_group(2016)
df_2017 = df_grouped.get_group(2017)

df_2015_grouped = df_2015.groupby(df_2015['MCI']).count()

```

```

df_2016_grouped = df_2016.groupby(df_2016['MCI']).count()
df_2017_grouped = df_2017.groupby(df_2017['MCI']).count()
plot = df_2015_grouped.iloc[:,0]
plot = pd.DataFrame(plot)
plot.columns = ['Number of Cases']
ax = plot.plot(kind='barh',figsize=(15,5),title='Number of Major Crimes Reported in Toronto in 2015')

```

```
df.head()
```

```
df.head()
```

```
#factorize occurenceday:
```

```
day_var = pd.factorize(df2['occurenceday'])
```

```
df2['occurenceday'] = day_var[0]
```

```
definition_list_day = day_var[1]
```

```
#factorize occurencedayofweek:
```

```
dayweek_var = pd.factorize(df2['occurencedayofweek'])
```

```
df2['occurencedayofweek'] = dayweek_var[0]
```

```
definition_list_day = dayweek_var[1]
```

```
#factorize division:
```

```
division_var = pd.factorize(df2['Division'])
```

```
df2['Division'] = division_var[0]
```

```
definition_list_division = division_var[1]
```

```
#factorize HOOD_ID:
```

```
hood_var = pd.factorize(df2['Hood_ID'])
```

```
df2['Hood_ID'] = hood_var[0]
```

```
definition_list_hood = hood_var[1]
```

```
#factorize occurencehour:
```

```

hour_var = pd.factorize(df2['occurrencehour'])
df2['occurrencehour'] = hour_var[0]
definition_list_hour = hour_var[1]

#factorize occurencedayofyear:
dayyear_var = pd.factorize(df2['occurencedayofyear'])
df2['occurencedayofyear'] = dayyear_var[0]
definition_list_dayyear = dayyear_var[1]

df2.head()
x = df2.drop(['MCI'],axis=1).values
y = df2['MCI'].values
X_train, X_test, y_train, y_test = train_test_split(x, y, test_size = 0.25, random_state = 21)
binary_encoder = OneHotEncoder(sparse=False, categories='auto')
encoded_X = binary_encoder.fit_transform(x)
X_train_OH, X_test_OH, y_train_OH, y_test_OH = train_test_split(encoded_X, y, test_size = 0.25,
random_state = 21)
classifier = RandomForestClassifier(n_estimators = 100, criterion = 'entropy', random_state = 42)
classifier.fit(X_train, y_train)
y_pred = classifier.predict(X_test)

print("Accuracy of Random Forest : ",accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test, y_pred))
print(classification_report(y_test,y_pred, target_names=definition_list_MCI))

classifier = RandomForestClassifier(n_estimators = 100, criterion = 'entropy', random_state = 42)
classifier.fit(X_train_OH, y_train_OH)
y_pred_OH = classifier.predict(X_test_OH)

print("Accuracy of Random Forest with OneHotEncoder : ",accuracy_score(y_test, y_pred))
print(confusion_matrix(y_test_OH, y_pred_OH))

```

```

print(classification_report(y_test_OH,y_pred_OH, target_names=definition_list_MCI))

import seaborn as sns
mci_monthwise = df.groupby(['occurrencemonth','MCI'],as_index=False).agg({'Total':'sum'})

plt.figure(figsize=(15, 7))
crime_count = mci_monthwise.pivot("MCI","occurrencemonth","Total" )

plt.yticks(rotation=1)
ax = sns.heatmap(crime_count,cmap="YlGnBu", linewidths=.5)
plt.title("Major Crime Indicators by Month",color = 'red',fontsize=14)
plt.show()

major_crime_indicator = df.groupby('MCI',as_index=False).size()
plt.subplots(figsize = (15, 6))
ct = major_crime_indicator.sort_values(ascending = False)
ax = ct.plot.bar()
ax.set_xlabel('Offence')
ax.set_ylabel('Total Number of Criminal Cases from 2014 to 2019')
ax.set_title('Crime Indicator',color = 'red',fontsize=25)
plt.show()

hour_crime_group = df.groupby(['occurrencehour','MCI'],as_index=False).agg({'Total':'sum'})
fig, ax = plt.subplots(figsize=(15,10))
hour_crime_group.groupby('MCI').plot(x="occurrencehour", y="Total", ax=ax,linewidth=5)
ax.set_xlabel('Hour')
ax.set_ylabel('Number of occurrences')
ax.set_title('Crime Types by Hour of Day in Toronto',color = 'red',fontsize=25)

df_g0 = df_2015.groupby(['Neighbourhood','MCI']).size().to_frame('count').reset_index()
df_g0 = df_g0.pivot(index='Neighbourhood',columns='MCI',values='count')

```

```
df_g0 = df_g0.dropna()
```

```
df_g = df_2016.groupby(['Neighbourhood','MCI']).size().to_frame('count').reset_index()
df_g = df_g.pivot(index='Neighbourhood',columns='MCI',values='count')
df_g = df_g.dropna()
```

```
df_g2 = df_2017.groupby(['Neighbourhood','MCI']).size().to_frame('count').reset_index()
df_g2 = df_g2.pivot(index='Neighbourhood',columns='MCI',values='count')
df_g2 = df_g2.dropna()
```

```
df_g0.head(10)
```

```
scaler = StandardScaler()
Sum_of_squared_distances0 = []
Sum_of_squared_distances = []
Sum_of_squared_distances2 = []
```

```
std_scale = scaler.fit(df_g0)
df_transformed0 = std_scale.transform(df_g0)
pca = PCA(n_components=3)
pca = pca.fit(df_transformed0)
X0 = pca.transform(df_transformed0)
K = range(1,15)
for k in K:
    km = KMeans(n_clusters=k)
    km = km.fit(df_transformed0)
    Sum_of_squared_distances0.append(km.inertia_)
```

```
std_scale = scaler.fit(df_g)
df_transformed = std_scale.transform(df_g)
pca = PCA(n_components=3)
```

```

pca = pca.fit(df_transformed)
X = pca.transform(df_transformed)
K = range(1,15)
for k in K:
    km = KMeans(n_clusters=k)
    km = km.fit(df_transformed)
    Sum_of_squared_distances.append(km.inertia_)

```

```

std_scale = scaler.fit(df_g2)
df_transformed2 = std_scale.transform(df_g2)
pca = PCA(n_components=3)
pca = pca.fit(df_transformed2)
X1 = pca.transform(df_transformed2)

```

```

K = range(1,15)
for k in K:
    km = KMeans(n_clusters=k)
    km = km.fit(df_transformed2)
    Sum_of_squared_distances2.append(km.inertia_)

```

```

plt.plot(K, Sum_of_squared_distances0, 'bx-')
plt.xlabel('k')
plt.ylabel('Sum_of_squared_distances')
plt.title('Elbow Method For Optimal k')
plt.show()

```

```

plt.plot(K, Sum_of_squared_distances, 'bx-')
plt.xlabel('k')
plt.ylabel('Sum_of_squared_distances')
plt.title('Elbow Method For Optimal k')

```

```
plt.show()
```

```
plt.plot(K, Sum_of_squared_distances2, 'bx-')
```

```
plt.xlabel('k')
```

```
plt.ylabel('Sum_of_squared_distances')
```

```
plt.title('Elbow Method For Optimal k')
```

```
plt.show()
```

```
for n_clusters in range(2,6):
```

```
    kmeans = KMeans(n_clusters=n_clusters , random_state=3425)
```

```
    cluster_labels = kmeans.fit_predict(X0)
```

```
    silhouette_avg = silhouette_score(X0, cluster_labels)
```

```
    print("For n_clusters =", n_clusters, "The average silhouette_score is :", silhouette_avg)
```

```
    sample_silhouette_values = silhouette_samples(X0, cluster_labels)
```

```
fig, (ax1, ax2) = plt.subplots(1, 2)
```

```
fig.set_size_inches(10, 5)
```

```
ax1.set_xlim([-0.1, 1])
```

```
ax1.set_ylim([0, len(X) + (n_clusters + 1) * 10])
```

```
y_lower = 10
```

```
for i in range(n_clusters):
```

```
    # samples belonging to
```

```
    # cluster i, and sort them
```

```
    ith_cluster_silhouette_values = \
```

```
        sample_silhouette_values[cluster_labels == i]
```

```
    ith_cluster_silhouette_values.sort()
```

```
    size_cluster_i = ith_cluster_silhouette_values.shape[0]
```

```

y_upper = y_lower + size_cluster_i

color = cm.nipy_spectral(float(i) / n_clusters)
ax1.fill_betweenx(np.arange(y_lower, y_upper),
                  0, ith_cluster_silhouette_values,
                  facecolor=color, edgecolor=color, alpha=0.7)

# Label
ax1.text(-0.05, y_lower + 0.5 * size_cluster_i, str(i))

# Computing the new y_lower
y_lower = y_upper + 10 # 10 for the 0 samples

ax1.set_title("The silhouette plot for the various clusters.")
ax1.set_xlabel("The silhouette coefficient values")
ax1.set_ylabel("Cluster label")

# vertical line
ax1.axvline(x=silhouette_avg, color="red", linestyle="--")

ax1.set_yticks([])
ax1.set_xticks([-0.2,-0.1, 0, 0.2, 0.4, 0.6, 0.8, 1])

# showing the actual cluster
colors = cm.nipy_spectral(cluster_labels.astype(float) / n_clusters)
ax2.scatter(X0[:, 0], X0[:, 1], marker='.', s=300, lw=0, alpha=0.7,
            c=colors, edgecolor='k')

# Labeling the clusters
centers = kmeans.cluster_centers_

# cluster centers

```



```

ax2.scatter(centers[:, 0], centers[:, 1], marker='o',
            c="white", alpha=1, s=200, edgecolor='k')

for i, c in enumerate(centers):
    ax2.scatter(c[0], c[1], marker='$%d$' % i, alpha=1,
                s=50, edgecolor='k')

ax2.set_title("The visualization of the clustered data.")
ax2.set_xlabel("Feature space for the 1st feature")
ax2.set_ylabel("Feature space for the 2nd feature")

plt.suptitle(("Silhouette analysis for KMeans clustering on sample data "
            "with n_clusters = %d" % n_clusters), fontsize=14, fontweight='bold')

plt.show()
ax1.legend()
ax2.legend()

#Plotting the cluster obtained using GMM
kmeans = KMeans(n_clusters=2, random_state=3425)
colors = ['green', 'blue']

plt.figure(figsize=(15, 5))

ax = plt.subplot(121)
kc = kmeans.fit(df_transformed0)
label = pd.DataFrame(kc.labels_)
df_result = pd.DataFrame(df_transformed0)
# label = label.sort_values(by=0)
df_result['label'] = label

```

```

scatter = plt.scatter(df_result[0],df_result[2],
                      c=list(label.iloc[:,0]), cmap=matplotlib.colors.ListedColormap(colors),s=50)
plt.title('K-Means Clustering Without PCA')
plt.xlabel('Assault')
plt.ylabel('Auto-Theft')
plt.colorbar(scatter)

ax = plt.subplot(122)
start = timeit.default_timer()
kc =kmeans.fit(X0)
stop = timeit.default_timer()
klabel0 = pd.DataFrame(kc.labels_)
df_result =pd.DataFrame(X0)
# klabel0 = klabel0.sort_values(by=0)
df_result['label']=klabel0
scatter = plt.scatter(df_result[0],df_result[2],
                      c = list(klabel0.iloc[:,0]), cmap=matplotlib.colors.ListedColormap(colors),s=50)
plt.title('K-Means Clustering With PCA')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.colorbar(scatter)
print("running time is :",stop-start)
plt.show()

colors = ['green','blue']
#Plotting the cluster obtained using GMM
kmeans = KMeans(n_clusters=2,random_state=342)
plt.figure(figsize=(15, 10))

ax = plt.subplot(221)

```

```

kc =kmeans.fit(df_transformed)

label = pd.DataFrame(kc.labels_)
df_result =pd.DataFrame(df_transformed)
label.sort_values(by=0)
df_result['label']=label
scatter = plt.scatter(df_result[0],df_result[2],
                      c = list(label.iloc[:,0]), cmap= matplotlib.colors.ListedColormap(colors),s=50)
plt.title('K-Means Clustering Without PCA :')
plt.xlabel('Assault')
plt.ylabel('Auto-Theft')
plt.colorbar(scatter)

ax = plt.subplot(222)
start = timeit.default_timer()
kc =kmeans.fit(X)
stop = timeit.default_timer()
klabel1 = pd.DataFrame(kc.labels_)
df_result =pd.DataFrame(X)
klabel1.sort_values(by=0)
df_result['label']=klabel1
scatter = plt.scatter(df_result[0],df_result[2],
                      c = list(klabel1.iloc[:,0]), cmap= matplotlib.colors.ListedColormap(colors),s=50)
plt.title('K-Means Clustering With PCA')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.colorbar(scatter)
print("running time is :",stop-start)
plt.show()

#Plotting the cluster obtained using GMM

```

```

kmeans = KMeans(n_clusters=2,random_state=342)
plt.figure(figsize=(15, 15))
ax = plt.subplot(321)
kc =kmeans.fit(df_transformed2)
label = pd.DataFrame(kc.labels_)
df_result =pd.DataFrame(df_transformed2)
label.sort_values(by=0)
df_result['label']=label
scatter = plt.scatter(df_result[0],df_result[2],
                      c = list(label.iloc[:,0]), cmap= matplotlib.colors.ListedColormap(colors),s=50)
plt.title('K-Means Clustering Without PCA')
plt.xlabel('Assault')
plt.ylabel('Auto-Theft')
plt.colorbar(scatter)
ax.legend()

ax = plt.subplot(322)
start = timeit.default_timer()
kc =kmeans.fit(X1)
stop = timeit.default_timer()
klabel2 = pd.DataFrame(kc.labels_)
df_result =pd.DataFrame(X1)
klabel2.sort_values(by=0)
df_result['label']=klabel2
scatter = plt.scatter(df_result[0],df_result[2],
                      c = list(klabel2.iloc[:,0]), cmap= matplotlib.colors.ListedColormap(colors),s=50)
plt.title('K-Means Clustering With PCA')
plt.xlabel('PC1')
plt.ylabel('PC2')
plt.colorbar(scatter)
print("running time is :",stop-start)

```

```

ax.legend()
plt.show()
ax.legend()

X_list = [X0, X, X1]
year_list = [2015,2016,2017]
for i,y in zip(X_list,year_list):
    from sklearn import metrics
    clustering = KMeans(n_clusters=2,random_state=3425).fit(i)
    labels = clustering.labels_
    print("Silhouette score(K-means) for year",y,"is",metrics.silhouette_score(i, labels,
metric='euclidean'))

neighborhoods0 = df_g0.index
neighborhoods0 = np.array(neighborhoods0)

neighborhoods = df_g.index
neighborhoods = np.array(neighborhoods)

neighborhoods2 = df_g2.index
neighborhoods2 = np.array(neighborhoods2)
#Display Neighborhoods
print('Crime Neighborhoods by K-Means in 2015')
print(neighborhoods0[np.where(klabel0[0]==1)])
kviolent15 = neighborhoods0[np.where(klabel0[0]==1)]
kviolent15s = neighborhoods0[np.where(klabel0[0]==0)]
kviolent15m = neighborhoods0[np.where(klabel0[0]==2)]

print('Crime Neighborhoods by K-Means in 2016')
print(neighborhoods[np.where(klabel1[0]==1)])
kviolent16 = neighborhoods[np.where(klabel1[0]==1)]

```

```

kviolent16s = neighborhoods[np.where(klabel1[0]==0)]
kviolent16m = neighborhoods[np.where(klabel1[0]==2)]

print('Crime Neighborhoods by K-Means in 2017')
print(neighborhoods2[np.where(klabel2[0]==1)])
kviolent17 = neighborhoods2[np.where(klabel2[0]==1)]
kviolent17s = neighborhoods2[np.where(klabel2[0]==0)]
kviolent17m = neighborhoods2[np.where(klabel2[0]==2)]

plt.figure(num=None, figsize=(10, 8))
plt.scatter("Long", "Lat", data = df, c = 'y',alpha = 0.1, edgecolor = 'black', s=2)
plt.grid()
plt.xlabel('long')
plt.ylabel('lat')
plt.title('Toronto Crime')
plt.tight_layout()
plt.axis('tight')
plt.show()

```

analysis.ipynb:

```

import warnings
import itertools

import pandas as pd
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import seaborn as sns
from tqdm import tqdm

```

```

import statsmodels.api as sm
from sklearn.metrics import r2_score
from fbprophet import Prophet

warnings.filterwarnings("ignore")
plt.style.use('fivethirtyeight')
matplotlib.rcParams['axes.labelsize'] = 16
matplotlib.rcParams['xtick.labelsize'] = 14
matplotlib.rcParams['ytick.labelsize'] = 14
matplotlib.rcParams['text.color'] = 'k'
matplotlib.rcParams['figure.figsize'] = (30,6)
matplotlib.rcParams['axes.titlesize'] = 20

def mapperDictGenerator(df, label):
    d= dict()
    factorized= pd.factorize(df[label])
    for i, _ in enumerate(factorized[1]):
        d[i]= _
    return factorized[0], d

df=pd.read_csv('MCI_2014_to_2019.csv')
df.head()

df.describe()

plt.figure(figsize=(30,10))
MCI= df.groupby('MCI',as_index=False).size()
assaultTypes= df[df.MCI=='Assault'].groupby('offence',
as_index=False).size()
autoTheftTypes= df[df.MCI=='Auto Theft'].groupby('offence',

```

```
as_index=False).size()
```

```
plt.subplot(221)
sns.barplot(x='MCI', y='size', data=MCI.sort_values(by='size',
ascending=False))
plt.title('Major Crime Indicator', fontsize='xx-large')
plt.xlabel('Types of Crime', fontsize='x-large')
plt.ylabel('Crime Count', fontsize='x-large')
```

```
plt.subplot(222)
sns.barplot(x='offence', y='size',
data=assaultTypes.sort_values(by='size', ascending=False))
plt.title('Offence Distribution by Assault', fontsize='xx-large')
plt.xlabel('Types of Offence', fontsize='x-large')
plt.ylabel('Offence Count', fontsize='x-large')
plt.xticks(rotation=90)
plt.show()
```

```
plt.figure(figsize=(30,8))
```

```
breakEnterTypes= df[df.MCI=='Break and Enter'].groupby('offence',
as_index=False).size()
robberyTypes= df[df.MCI=='Robbery'].groupby('offence',
as_index=False).size()
theftOverTypes= df[df.MCI=='Theft Over'].groupby('offence',
as_index=False).size()
```

```
plt.subplot(131)
sns.barplot(x='offence', y='size',
data=breakEnterTypes.sort_values(by='size', ascending=False))
plt.title('Offence Distribution by Break and Enter', fontsize='xx-large')
```



```

plt.xlabel('Types of Offence', fontsize='x-large')
plt.ylabel('Offence Count', fontsize='x-large')
plt.xticks(rotation=90)

plt.subplot(132)
sns.barplot(x='offence', y='size',
data=robberyTypes.sort_values(by='size', ascending=False))
plt.title('Offence Distribution by Robbery', fontsize='xx-large')
plt.xlabel('Types of Offence', fontsize='x-large')
plt.ylabel('Offence Count', fontsize='x-large')
plt.xticks(rotation=90)

plt.subplot(133)
sns.barplot(x='offence', y='size',
data=theftOverTypes.sort_values(by='size', ascending=False))
plt.title('Offence Distribution by Theft Over', fontsize='xx-large')
plt.xlabel('Types of Offence', fontsize='x-large')
plt.ylabel('Offence Count', fontsize='x-large')
plt.xticks(rotation=90)

plt.show()

df.reporteddate= pd.to_datetime(df.reporteddate)
df['reportOnlyDate']= df.reporteddate.dt.date
df.occurrenceDate= pd.to_datetime(df.occurrenceDate)
df['occurrenceOnlyDate']= df.occurrenceDate.dt.date
df['delayInReporting']= (df.reportOnlyDate -
df.occurrenceOnlyDate).dt.days

df['count']= 1
mci_monthwise =

```

```

df.groupby(['occurrenceday','MCI'],as_index=False).agg({'count':'sum'})

mci_distribution =
df.groupby(['occurrenceyear','occurrencemonth'],as_index=False).agg({'count':'sum'})

mci_distribution['occurrenceyear']=
mci_distribution['occurrenceyear'].astype('int').astype('str')
mci_distribution['monthYear']= mci_distribution['occurrencemonth'] +',
'+ mci_distribution['occurrenceyear']

plt.grid('on')
plt.plot(mci_distribution['monthYear'], mci_distribution['count'])
plt.xlabel('Month Stream')
plt.ylabel('Count of Crime')
plt.title('Time Series Distribution of Crime [Monthwise]')
plt.xticks(np.arange(0, mci_distribution['monthYear'].shape[0], 3),
rotation=90)
plt.show()

occurrenceDateVsMciCount=
df.groupby(['occurrenceOnlyDate','MCI'],as_index=False).agg({'count':'sum'})

trimmedOccuranceDateVsMciCount=
occurrenceDateVsMciCount[occurrenceDateVsMciCount.occurrenceOnlyDate > pd.to_datetime('2014-12-31').date()]

# occurrenceDateCount=
df.groupby(['occurrenceOnlyDate'],as_index=False).agg({'count':'sum'})
occurrenceDateCount=
df.groupby('occurrenceOnlyDate')['count'].sum().reset_index()
trimmedOccuranceDateCount=
occurrenceDateCount[occurrenceDateCount.occurrenceOnlyDate >

```

```

pd.to_datetime('2014-12-31').date()]
    trimmedOccuranceDateCount['occurrenceOnlyDate']=
pd.to_datetime(trimmedOccuranceDateCount.occurrenceOnlyDate)
    trimmedOccuranceDateCount.set_index('occurrenceOnlyDate',
inplace=True)

decomposition=
sm.tsa.seasonal_decompose(trimmedOccuranceDateCount[trimmedOccur
anceDateCount.index > pd.to_datetime('2018-12-31')], model='additive')

plt.rcParams['figure.figsize'] = 24, 8
decomposition.plot()

plt.show()

sns.distplot(trimmedOccuranceDateCount['count'])
plt.show()

trainData=
trimmedOccuranceDateCount[trimmedOccuranceDateCount.index <
pd.to_datetime('2018-12-31')]
testData=
trimmedOccuranceDateCount[trimmedOccuranceDateCount.index >
pd.to_datetime('2018-12-31')]

y= trimmedOccuranceDateCount['count'].resample('MS').mean()

## **SARIMA** Time Series Forecasting
p = d = q = range(0, 2)
pdq = list(itertools.product(p, d, q))
seasonal_pdq = [(x[0], x[1], x[2], 12) for x in list(itertools.product(p, d,

```

q))]

```
print('Examples of parameter combinations for Seasonal ARIMA...')
print('SARIMAX: {} x {}'.format(pdq[1], seasonal_pdq[1]))
print('SARIMAX: {} x {}'.format(pdq[1], seasonal_pdq[2]))
print('SARIMAX: {} x {}'.format(pdq[2], seasonal_pdq[3]))
print('SARIMAX: {} x {}'.format(pdq[2], seasonal_pdq[4]))

resultDict= dict()

for param in pdq:
    for param_seasonal in seasonal_pdq:
        try:
            mod= sm.tsa.statespace.SARIMAX(trainData, order=param,
seasonal_order=param_seasonal, enforce_stationarity=False,
enforce_invertibility=False)
            results= mod.fit()
            print('ARIMA{}x{}12 - AIC:{}'.format(param, param_seasonal,
results.aic))
            resultDict['ARIMA{}x{}12'.format(param, param_seasonal)]=
results.aic
        except:
            continue

minIndexARIMA=
list(resultDict.keys())[list(resultDict.values()).index(min(resultDict.values
())))]

print('MIN INDEX: {}'.format(minIndexARIMA))

crimeforecast=results.forecast(steps=365)[0]

testData
```

```
SARIMAXmodel= sm.tsa.statespace.SARIMAX(trainData, order=(1, 1,
1), seasonal_order=(0, 1, 1, 12), enforce_stationarity=False,
enforce_invertibility=False)
```

```
    SARIMAXresults= SARIMAXmodel.fit()
    print(SARIMAXresults.summary().tables[1])
    SARIMAXresults.plot_diagnostics(figsize=(20, 8))
    plt.show()
    pred_uc= SARIMAXresults.get_forecast(steps=365)
    pred_ci= pred_uc.conf_int()
    ax = y.plot(label='observed', figsize=(20, 7))
    pred_uc.predicted_mean.plot(ax=ax, label='Forecast')
    ax.fill_between(pred_ci.index,
                    pred_ci.iloc[:, 0],
                    pred_ci.iloc[:, 1], color='k', alpha=.25)
    ax.set_xlabel('Date')
    ax.set_ylabel('Crime Count')
    plt.legend()
    plt.show()
```

```
testData['predValues']= pred_uc.predicted_mean.astype('int')
```

```
    testData['lowerCount']= pred_ci[pred_ci.index >=
testData.index.min()]['lower count']
    testData['upperCount']= pred_ci[pred_ci.index >=
testData.index.min()]['upper count']
```

```
    mse= round((((testData['count'] - testData['predValues'])**2).mean(), 2)
    rmse= round(np.sqrt(mse), 2)
    ubmse= round((((testData['upperCount'] - testData['count'])**2).mean(),
2)
```

```

ubrmse= round(np.sqrt(ubmse), 2)
lbmse= round((((testData['lowerCount'] - testData['count'])**2).mean(),
2)
lbrmse= round(np.sqrt(lbmse), 2)

print('Mean Square Error: {}'.format(mse))
print('Root Mean Square Error: {}'.format(rmse))
print('Mean Square Error with Upper Envelop: {}'.format(ubmse))
print('Root Mean Square Error with Upper Envelop:
{}'.format(ubrmse))
print('Mean Square Error with Lower Envelop: {}'.format(lbmse))
print('Root Mean Square Error with Lower Envelop: {}'.format(lbrmse))
print()
print(testData.head())

```

CHAPTER 7

SNAPSHOTS

General:

This project detects the crimes , implemented using python and crime data as an input.

SNAPSHOTS

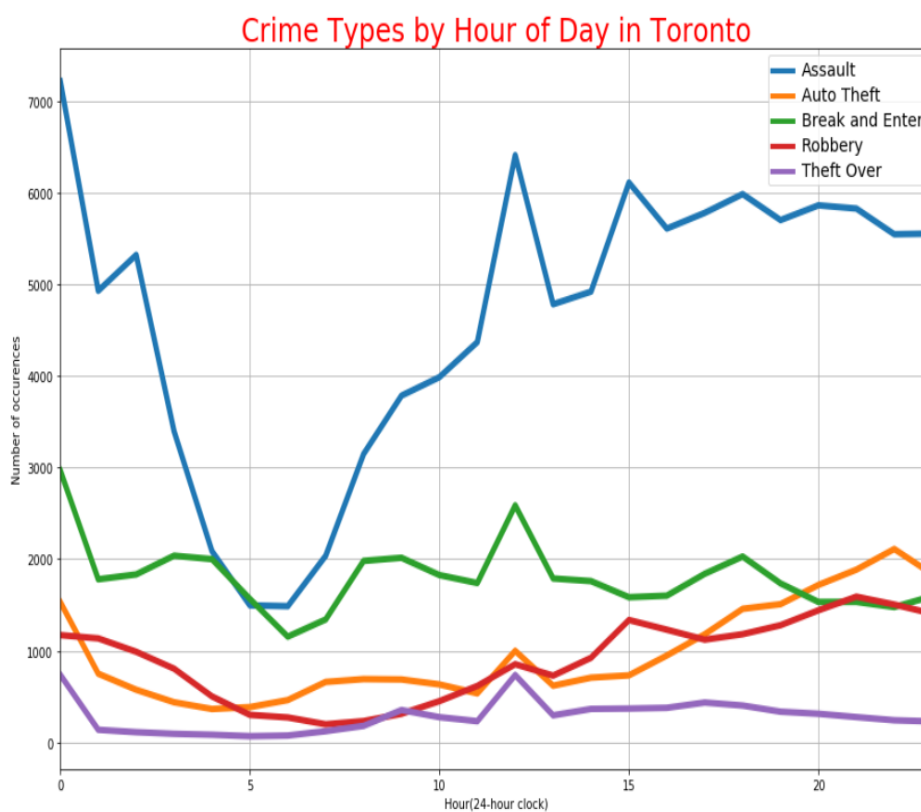


Fig 7.1 Crime types by hour of day in Toronto

In figure 7.1, we can see that the final result of the project detects the types of crimes which are shown with the help of the graph and we can see that the model detects the different types of the crimes.

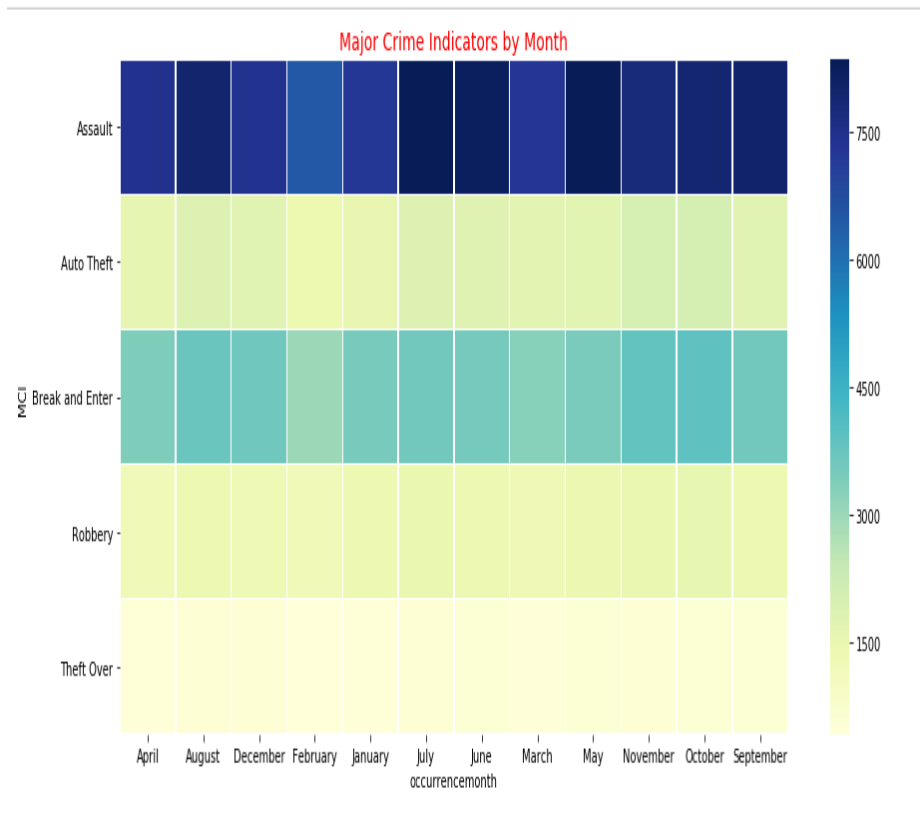


Fig 7.2 Major crime indicators by month

In figure 7.2, We can see that the final result of the project detects the crimes which are shown with the help of the green rectangular boxes while the non crime hotspots are shown with the pale pink rectangular boxes .

CHAPTER 8

SOFTWARE TESTING

8.1 GENERAL

The purpose of testing is to discover errors. Testing is the process of trying to discover every conceivable fault or weakness in a work product. It provides a way to check the functionality of components, sub assemblies, assemblies and/or a finished product. It is the process of exercising software with the intent of ensuring that the Software system meets its requirements and user expectations and does not fail in an unacceptable manner. There are various types of test. Each test type addresses a specific testing requirement.

8.2 DEVELOPING METHODOLOGIES

The test process is initiated by developing a comprehensive plan to test the general functionality and special features on a variety of platform combinations. Strict quality control procedures are used. The process verifies that the application meets the requirements specified in the system requirements document and is bug free. The following are the considerations used to develop the framework from developing the testing methodologies.

8.3 Types of Tests

8.3.1 Unit testing

Unit testing involves the design of test cases that validate that the internal program logic is functioning properly, and that program input produce valid outputs. All decision branches and internal code flow should be validated. It is the testing of individual software units of the application .it is done after the completion of an individual unit before integration. This is a structural testing, that relies on knowledge of its construction and is invasive. Unit tests perform basic tests at component level and test a specific business process, application, and/or system configuration. Unit tests ensure that each unique path of a business process performs accurately to the documented specifications and contains clearly defined inputs and expected results.

8.3.2 Functional test

Functional tests provide systematic demonstrations that functions tested are available as specified by the business and technical requirements, system documentation, and user manuals.

Functional testing is centered on the following items:

- Valid Input : identified classes of valid input must be accepted.
- Invalid Input : identified classes of invalid input must be rejected.
- Functions : identified functions must be exercised.
- Output : identified classes of application outputs must be exercised.
- Systems/Procedures : interfacing systems or procedures must be invoked.

8.3.3 System Test

System testing ensures that the entire integrated software system meets requirements. It tests a configuration to ensure known and predictable results. An example of system testing is the configuration oriented system integration test. System testing is based on process descriptions and flows, emphasizing pre driven process links and integration points.

8.3.4 Performance Test

The Performance test ensures that the output be produced within the time limits, and the time taken by the system for compiling, giving response to the users and request being send to the system for to retrieve the results.

8.3.5 Integration Testing

Software integration testing is the incremental integration testing of two or more integrated software components on a single platform to produce failures caused by interface defects. The task of the integration test is to check that components or software applications, e.g. components in a software system or – one step up – software applications at the company level – interact without error.

8.3.6 Acceptance Testing

User Acceptance Testing is a critical phase of any project and requires significant participation by the end user. It also ensures that the system meets the functional requirements.

Acceptance testing for Data Synchronization:

- The Acknowledgements will be received by the Sender Node after the Packets are received by the Destination Node
- The Route add operation is done only when there is a Route request in need
- The Status of Nodes information is done automatically in the Cache Updation process

Build the test plan

Any project can be divided into units that can be further performed for detailed processing. Then a testing strategy for each of this unit is carried out. Unit testing helps to identity the possible bugs in the individual component, so the component that has bugs can be identified and can be rectified from errors

CHAPTER 9

CONCLUSION

The proposed crime prediction and analysis system demonstrates the transformative potential of machine learning in enhancing public safety and optimizing law enforcement strategies. By integrating advanced techniques such as K-Means clustering, ARIMA, SARIMA, Random Forest, and PCA, the system effectively identifies crime hotspots, predicts future crime trends, and classifies crime types. These insights enable proactive decision-making, efficient resource allocation, and targeted interventions, shifting the focus of law enforcement from reactive to data-driven policing.

The use of spatial, temporal, and categorical analyses ensures a comprehensive understanding of crime patterns. By identifying high-risk areas and forecasting crime trends, the system allows law enforcement agencies to prioritize efforts and allocate resources where they are most needed. Additionally, its ability to visualize data using heatmaps and graphs improves decision-making and raises public awareness, fostering safer communities.

In the future, the system can be enhanced by integrating real-time data from IoT devices and surveillance systems to provide live updates and improve responsiveness. Advanced deep learning models, such as Convolutional Neural Networks (CNNs) and Recurrent Neural Networks (RNNs), could be employed for more accurate predictions and deeper insights. Incorporating socioeconomic and behavioral data would also offer a more holistic approach to understanding and preventing crime.

REFERENCES

- [1] Mohd, Safiya and Hithasri Muthyala. "Crime Data Visualization and Forecasting Using SARIMA Algorithm." (2023).
- [2] M. Feng et al., "Big Data Analytics and Mining for Effective Visualization and Trends Forecasting of Crime Data," in IEEE Access, vol. 7, pp. 106111-106123, 2019
- [3] G. Jha, L. Ahuja and A. Rana, "Criminal Behaviour Analysis and Segmentation using K-Means Clustering," 2020 8th International Conference on Reliability, Infocom Technologies and Optimization (Trends and Future Directions) (ICRITO), Noida, India, 2020, pp. 1356-1360
- [3] U. M. Butt, S. Letchmunan, F. H. Hassan, M. Ali, A. Baqir and H. H. R. Sherazi, "Spatio-Temporal Crime HotSpot Detection and Prediction: A Systematic Literature Review," in IEEE Access, vol. 8, pp. 166553-166574, 2020
- [4] V. Mandalapu, L. Elluri, P. Vyas and N. Roy, "Crime Prediction Using Machine Learning and Deep Learning: A Systematic Review and Future Directions," in IEEE Access, vol. 11, pp. 60153-60170, 2023
- [5] Noor, T.H.; Almars, A.M.; Alwateer, M.; Almaliki, M.; Gad, I.; Atlam, E.-S. SARIMA: A Seasonal Autoregressive Integrated Moving Average Model for Crime Analysis in Saudi Arabia. Electronics 2022
- [6] N. Thoiba Singh, M. Mehra, I. Verma, N. Singh, D. Gandhi and M. Ahmad Alladin, "Advancing Crime Analysis and Prediction: A Comprehensive Exploration of Machine Learning Applications in Criminal Justice," 2024 2nd International Conference on Intelligent Data Communication Technologies
- [7] P. Chen, H. Yuan and X. Shu, "Forecasting Crime Using the ARIMA Model," 2008 Fifth International Conference on Fuzzy Systems and Knowledge Discovery, Jinan, China, 2008, pp. 627-630
- [8] A. S. R, D. Nidhi, D. S. C and K. Sowjanya K, "Crime Forecasting : A Theoretical Approach," 2022 IEEE 7th International Conference on Recent Advances and Innovations in Engineering (ICRAIE), MANGALORE, India, 2022, pp. 37-41
- [9] T. Reddi, C. Kusuma and S. Parvin, "Mapping Crime Dynamics: Integrating Textual, Spatial, and Temporal Perspectives," 2024 IEEE 15th Annual Ubiquitous Computing, Electronics & Mobile Communication Conference (UEMCON), Yorktown Heights, NY, USA, 2024, pp. 757-763
- [10] V. Mandalapu, L. Elluri, P. Vyas and N. Roy, "Crime Prediction Using Machine Learning and Deep Learning: A Systematic Review and Future Directions," in IEEE Access, vol. 11, pp. 60153-60170, 2023
- [11] Z. Sabakhtarishvili, S. Panday, C. Jensen and R. Ghosh, "Crime Prediction Using Machine Learning: The Case of The City of Little Rock," 2023 5th International Congress on Human-Computer Interaction, Optimization and Robotic Applications (HORA), Istanbul, Turkiye, 2023, pp. 1-6